

An Improved GPU-Accelerated Heuristic Technique Applied to the Capacitated Vehicle Routing Problem

Marwan F. Abdelatti

Department of Industrial Engineering
University of Rhode Island
Kingston, Rhode Island
mabdelrazik@uri.edu

Manbir S. Sodhi

Department of Industrial Engineering
University of Rhode Island
Kingston, Rhode Island
sodhi@uri.edu

ABSTRACT

The capacitated vehicle routing problem (CVRP) is a well-known NP-hard combinatorial problem. Genetic algorithms (GAs) are often used for solving CVRPs. However, the computational effort required to find a feasible solution becomes problematic for very large instances. Parallel-computation technology can significantly improve the performance of CVRP solution algorithms to deal with large problem sets, especially when using GAs. In this paper, an improved genetic algorithm is designed to be entirely executed on an NVIDIA GPU, taking advantage of the special CUDA GPU architecture to solving the CVRP. By distributing array elements over the GPU grid and using GPU kernel functions, the proposed algorithm successfully provides high-quality solutions within reasonable computational times, and near-optimal solutions for smaller benchmark problems. Within this framework, we address the execution speed problem in CVRPs by developing an algorithm that is entirely running on an NVIDIA GPU, investigate how to incorporate local search algorithms with the GA, and develop comparisons between the algorithm performance on both the CPU and the GPU. The utility of improved local search algorithms in the overall performance of the GA is also reported.

CCS CONCEPTS

• **Mathematics of computing** → *Evolutionary algorithms*; • **Computing methodologies** → *Parallel computing methodologies*; • **Applied computing** → *Operations research*;

KEYWORDS

Vehicle routing problem, genetic algorithms, local search, GPU, parallelization

ACM Reference Format:

Marwan F. Abdelatti and Manbir S. Sodhi. 2020. An Improved GPU-Accelerated Heuristic Technique Applied to the Capacitated Vehicle Routing Problem. In *Genetic and Evolutionary Computation Conference (GECCO '20)*, July 8–12, 2020, Cancún, Mexico. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3377930.3390159>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '20, July 8–12, 2020, Cancún, Mexico

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7128-5/20/07...\$15.00

<https://doi.org/10.1145/3377930.3390159>

1 INTRODUCTION

Vehicle routing and scheduling problems occupy a special place in distribution management. They are among the most important and challenging optimization problems in the field of operations research because of their complexity and wide applicability. They can model many real-life applications such as assembly lines, postal service planning, logistics distribution systems, waste collection, etc. The vehicle routing problem (VRP) is an NP-hard combinatorial problem for determining optimal delivery/collection routes through a set of locations, subject to some constraints [14]. The capacitated VRP (CVRP) is the most common and the simplest form of VRP where the total demands of the customers assigned to a vehicle cannot exceed the vehicle capacity, and the customers must be visited only once. There are many methods used for solving the CVRP; some are exact methods, and the others are heuristic.

Exact methods mainly depend on direct tree searches, dynamic programming, or linear programming [29, 30]. These methods can provide optimal or near-optimal solutions but require extended computational resources and this limits the problem size they can solve. There are numerous examples of such methods including but not limited to branch-and-bound (B&B), branch-and-cut (B&C) and dynamic programming [16]. The most sophisticated exact methods nowadays can solve problems up to a little more than one hundred customers [45]. One of such method is presented in [4] and employs a partitioning formulation and additional cuts. However, practical problems consist of hundreds or even thousands of customers. For such problems, an exact solution within a reasonable amount of time is hard to obtain [17].

Heuristic methods, on the other hand, can provide high quality near-optimal solutions within acceptable computational times. Many heuristic techniques generate solutions for the CVRP in a progressive manner [29, 45]. Examples of such techniques include the Clarke and Wright algorithm [12], Tabu Search algorithm [23], local search algorithm (2-Opt) [36], and genetic algorithms (GA) [3]. Genetic Algorithms (GA) are a type of heuristic method that utilize a number of basic reproduction features existing in nature such as: selection, mating, and mutation processes [5]. They are distinguished from other heuristic methods by the fact that they operate on a population of potential solutions which increases the probability of finding a good solution and leads to having them used effectively in many VRP scenarios [3, 11, 15, 19, 21, 26, 35, 37, 39, 40]. Extensive research efforts to incorporate exact methods with GAs have been reported [3, 21, 37, 41]. However, GAs require long execution times, and parallelizing algorithms on multi-core processing units are one approach for finding high quality solutions in a feasible amount of

time. With recent hardware advances of graphics processing units (GPUs), these are also being considered for GA applications.

GPUs are multiprocessor devices that can run a large number of instructions in parallel at a high execution speed. They are a key utility in the revolutionary advancement of the processing speed occurring nowadays.

In this paper, a new GA algorithm is proposed to find the best route (with shortest distance) for CVRPs. This algorithm is specifically designed to be entirely executed on the GPU to take advantage of the special NVIDIA compute unified device architecture (CUDA) for parallel computations with very short execution times. The diversity of the population is maintained along the evolution process to prevent the algorithm from getting stuck into a local minimum. Local search algorithms (i.e., 2-opt and nearest neighbor) are utilized to study their effect on the quality of the solutions. They are employed to optimize every single route in each solution in the GA population. Within this framework, we address the execution speed problem in VRPs by developing an algorithm that entirely runs on an NVIDIA GPU and provide comparisons between the algorithm performance on both CPU and GPU in tables and plots. Moreover, we improve some best-documented values of existing VRP benchmark problems, and investigate the effectiveness of utilizing local search algorithms on the overall performance of the GA. The rest of the paper is organized as follows; Section 2 briefly reviews existing works related to VRP, GA, and parallelization. The methodology of the proposed algorithm is introduced in Section 3, including an illustration of the GPU technology and the implementation of the proposed algorithm on the GPU. Section 4 discusses the results of applying the algorithm on benchmark problems and compares between CPU and GPU runs. Some conclusions are stated at the end of the paper.

2 RELATED WORK

A significant body of research on algorithms for solving VRPs is documented. Broadly speaking, these algorithms can be divided into exact methods and heuristic based approaches. Exact methods guarantee that the optimal solution is reached either by searching exhaustively over all possible solutions or by eliminating solutions that lead to non-optimal solutions. Heuristic methods, on the other hand, perform a relatively limited exploration of the search space and typically produce good quality solutions within modest computing times [34]. Genetic algorithms (GAs) are a type of heuristics that operate with a population of solutions to making small changes (mutation) and mixing parts (crossover) in consideration of quality objectives to guide its search toward the optimal solutions [33]. Moreover, most effective GAs are hybridized with local search algorithms [44]. This approach may be viewed as a multi-start local search method, with starting points obtained through a sampling of the search space provided by the genetic algorithm[44].

Some of these algorithms are implemented on traditional computers involving single or multiple CPUs, while others are partially or entirely implemented on the GPU. In [3], a hybrid Genetic Algorithm was applied to a CVRP problem. The chromosomes were represented such that each individual solution specifies the vehicle number to which each customer was assigned. Even though the

chromosome representation guaranteed a fixed length for each single solution which simplifies the implementation, the algorithm produced routes with higher excessive demands than the vehicle's capacity. Another algorithm was introduced in [37] where a 2-opt local search algorithm was utilized with the GA. The chromosomes were represented as a sequence of customers without depot delimiters and a split algorithm was applied to consider the capacity constraint. This chromosome representation has been found more efficient and is adopted by most of the subsequent works on GAs for VRPs in the literature. Duplicate-solutions (clones) were prohibited by a special algorithm. The down side of this algorithm was in the huge computational load (almost 95% of CPU time) consumed by the local search and the clone-restricting algorithms [37]. The work in [20] proposed a more efficient method to restrict cloning but the solutions were still not good enough. An edit distance measure was employed in [21, 41] to improve the clone-restricting algorithm but it involved extended execution times.

Preserving the population diversity is crucial to any algorithm involving GAs. It helps avoiding premature convergence and balances between exploration and exploitation of the search space, yet is very computationally expensive. The high computational cost limits the size of the VRP problems the algorithms deal with. Therefore, many research efforts that utilize GPUs for VRP problems are conducted to speed up the algorithm execution speed. [6] designed a genetic algorithm implemented on the GPU for the Dynamic VRP (DVRP) - which involves finding VRP solutions with some demands that are revealed after the tour is in progress. It was effective to find good solutions for problems up to 3000 nodes with a speed 35 times faster than the CPU implementation. However, the method assumed that the routes were already planned and targeted only to insert new requests in these routes then genetic operators optimize the resulting solution.

A CPU-GPU integrated perturbation strategy and four different neighborhood local searches implemented purely in GPUs were introduced in [13] for single vehicle routing problem with deliveries and selective pickups (SVRPDSP). The initial solution is generated by two mathematical programming solvers, providing the search algorithm with good starting solutions. The algorithm was able to achieve nearly 15 times speedup over a pure sequential version but a considerable part of the algorithm was executed on the CPU affecting the total execution time. The algorithm was not tested on problems larger than 101 customer nodes. The parallel GPU execution in [46] was only applied to the cost function calculation for CVRP and showed a speed up in the cost calculation function of about 1,000 times than the CPU implementation. It did not, however, report the impact of this implementation on the overall performance of the algorithm.

A parallel GA running on the GPU for the Min-Max Multi-depot VRP (MDVRP) was proposed in [32]. By assigning the computing tasks for each point in the chromosome (i.e., gene) to independent threads, the algorithm was able to process all the GA operations in parallel. Although the experimental results showed a significant reduction in the computing time that was about 1,000 times less than the CPU implementation for a problem with 200 nodes, the quality of the solution has not been discussed limiting any conclusions on the effectiveness of the proposed algorithm. Also, the initialization

of the population was processed on the CPU which affected the execution speed. The work in [43] utilized GPUs to run a Tabu Search (TS) algorithm for the Distance-constrained VRP and compared it with a CPU version. Even though its computational complexity was higher, the parallel algorithm was able to outperform the classic TS algorithm in terms of speed. However, the TS algorithm is very crude and more evolved models were required for more complex problems [43].

3 PROPOSED ALGORITHM

We introduce a new hybrid algorithm to find the best route with the shortest distance for the capacitated vehicle routing problem. This algorithm is specifically designed to be entirely executed on the GPU to take advantage of the special NVIDIA CUDA architecture for parallel computations. By distributing array elements over the GPU grid and using GPU kernel functions, the proposed algorithm provides high-quality solutions at a very high computational speeds, and near-optimal solutions for smaller benchmark problems. Before delving into the details of proposed algorithm, the following basic definitions are established.

3.1 Basic Notations and Definitions

The following notations are used in the paper. $\mathbb{R}$ and $\mathbb{R}_+$ denote the sets of real and positive real numbers respectively. Similarly, $\mathbb{I}$ and $\mathbb{I}_+$ represent the sets of integers and positive integers respectively. A set of numbers is represented by $\{n_1, n_2, \dots, n_k\}$ whereas a set of tuples is represented by $\{(m_1, m_2), (m_3, m_4), \dots, (m_p, m_q)\}$. Based on these notations, a particular CVRP can be defined as:

Definition 3.1. Given a depot node $\{0\}$ at location (x_0, y_0) and a set of n customer nodes $C = \{1, 2, 3, \dots, n\}$, $n \in \mathbb{I}_+$ at locations $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)$ where $x_0, \dots, x_n, y_0, \dots, y_n \in \mathbb{R}$, there exists an undirected graph $\mathcal{G} = (\mathcal{N}, \mathcal{A})$, that can be established where $\mathcal{N} = \{0\} \cup C$ and $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}, i < j\}$ is a set of edges/connections between the nodes. Each customer i orders a non-negative demand $d_i \in \mathbb{R}_+ \forall i$. It has to be noted that this demand cannot be divided and hence every customer can be visited only once. The cost c_{ij} associated to each edge $(i, j) \in \mathcal{A}$ can be defined as the Euclidean distance between node i and node j given as:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1)$$

It is required to find the optimal solution l^* that has the minimum cost, i.e.,

$$l^* = \underset{l \in \mathcal{P}}{\operatorname{argmin}} \sum_{i,j=0}^n c_{ij}, \quad \forall i \neq j \quad (2)$$

where $\mathcal{P}$ is the set of all feasible solutions for the problem.

For simplicity, we assume that the vehicles have identical capacities Q where $Q \geq d_i \forall i \in C$ so that any vehicle can be assigned to any route available in the solution to avoid more complexity of the problem. The total cost $f_{l,t}$ of an individual solution $l_{i,t}$ is the sum of the costs c_{ij} associated with all the edges in it i.e.,

$$f_{l,t} = \sum_{i,j=0}^n c_{ij}, \quad \forall i \neq j \quad (3)$$

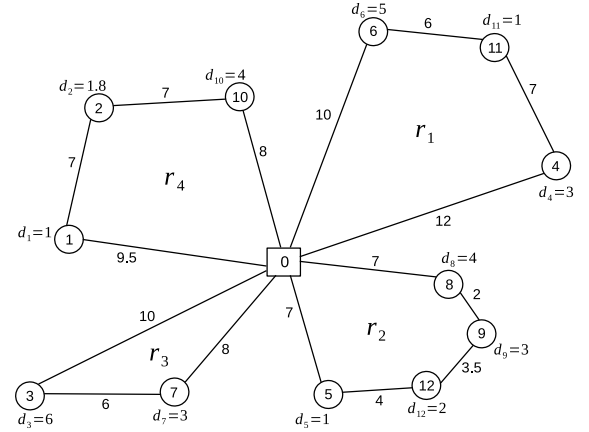


Figure 1: An example solution of a CVRP

Figure 1 illustrates the capacitated VRP problem by an example where the demand d_i of each of the 12 customers as well as the cost of the connections are shown. Four vehicles are available with capacity $Q = 10$. The feasible routes are represented by r_1, r_2, r_3 , and r_4 with $r_i \in \mathcal{A} \forall i$. For example, $r_1 = \{(0, 4), (4, 11), (11, 6), (6, 0)\}$ with a cost of 35 and load of $9 < Q$. The set $l = \{r_1 \cup r_2 \cup r_3 \cup r_4\} \in \mathcal{P}$ represents an example solution of this particular problem with a total cost of 114, this is not necessarily the optimal solution.

3.2 Algorithm Description

We incorporate a GA and a 2-opt local search into a hybrid algorithm to solve the CVRP defined in Definition (3.1). At each iteration t , population $P_t \subset \mathcal{A}$ includes all feasible solutions l_t where each row in P_t is a single solution. These solutions are called individuals and are represented as a sequence of customer nodes $\in C$ delimited by the depot node 0 to count for the capacity constraint. The number of individuals in population P_t is called the population size (*popSize*). The population size is predefined and is fixed along all iterations. At the beginning, the population P_0 is initiated by random individuals.

Evolutionary processes like cross-over and mutation are applied to each individual $l_{i,t} \forall i = \{1, \dots, \text{popSize}\}$ to update the population at each iteration $t > 0$ to form P_t . The cost value of each individual is computed when the individual is produced based on Equation 3 and the cost table which is generated by the algorithm. The 2-opt algorithm is used for every route inside each individual as a separate travelling salesman problem (TSP).

Two identical algorithms have been developed in this work, one is running on the GPU and the other is running on the CPU for performance comparison purposes. The two algorithms are identical except for the way the functions are coded to run properly on the corresponding hardware.

3.3 NVIDIA GPU Architecture

In recent years, much has been made in computers industry that caused a widespread shift to parallel computing. Nowadays, most consumer computers are shipped with graphics processing units

(GPUs). The rise of GPU computing introduced a new concept of performing general-purpose computations on it. In 2006, NVIDIA unveiled their first GPU with CUDA architecture, namely, GeForce 8800 GTX. It has been provided by many features that were not existing in GPUs before such as the unified shader pipeline which obviated the need for vertex and pixel shaders that are used to exist before, arithmetic logic units (ALU) complied with IEEE requirements for single-precision floating-point arithmetic, instruction set for general computation rather than specific graphics types, and a shared memory [42]. Thanks to this technology, a standard computational representation have been used and a variety of common programming languages (e.g., C, C++, and Python) have been involved.

As a result of these improvements, many industries and applications have been successfully developed by counting on CUDA technology. These benefits included performance-per-dollar and performance-per-watt improvement compared to similar implementations built on traditional central processing units (CPUs) [38]. Some successful implementations on GPUs included medical imaging, computational fluid dynamics, environmental science, and the revolutionary machine learning frameworks like Tensorflow and pyTorch [1, 24, 27].

A typical NVIDIA GPU consists of a virtual container called (grid) that includes several processors called Streaming Multiprocessors (SMs) that contain hundreds of cores (SPs). These cores are responsible for running the threads that execute the same code in parallel. A block is a unit consisting of one or more warps of 32 threads each. The number of threads can greatly exceed the number of cores thanks to the automatic thread scheduling policies such as: independent thread scheduling, Round Robin (RR) - Instructions, Thread block-based criticality aware warp scheduling, etc. [28]. Functions executed on the GPU (kernels) can be called either from the GPU by another kernel or from a CPU (host) function by passing the number of blocks and threads that can be configured manually.

GPUs have a special memory structure. The main memory type used by our algorithm is the global memory that is analogous to RAM for CPUs. It is the largest memory in a GPU with size up to 16 GB in some products. It is accessible by all threads as well as the CPU. An illustration of the basic architecture of a typical GPU is shown in Figure 2 where a grid contains several blocks or SMs with each consisting of several cores (SPs), a single shared memory which is a small memory accessed by the threads of a single block at a very high speed, and L1 and L2 cache memories that have much higher bandwidth than their CPU counterparts. All SMs share a single global memory, and an L2 cache [8].

3.4 Algorithm Implementation on GPU

In this implementation, the device grid is arranged as a 2D array of $B \times B$ blocks with each block being a 2D array of $T \times T$ threads where B and T are integer numbers. The grid, therefore, has a total of $(B * T)^2$ threads running in parallel. Every input array (e.g., the population P_t) is mapped into the grid in such a way that its elements are distributed over the threads that perform computations separately. This, in turn, facilitates high throughput memory transfers in the GPU.

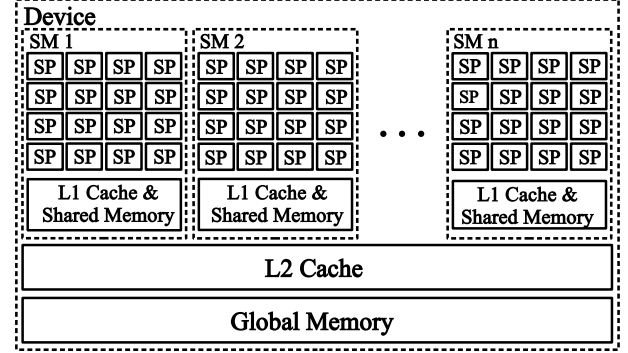


Figure 2: A typical GPU architecture.

Several kernels running on the GPU are devoted to perform different functions of the algorithm. These functions include the random initialization, cost calculation, cross over, capacity constraints, 2-opt local search. After reaching a specific number of generations given by the user, the GPU passes the final solution to the CPU that prints out the best solution as well as the total running time and the time elapsed to produce one generation.

The kernels in the proposed algorithm can be divided into two types based on how they deal with the input arrays. The first type is a scalar function where a single thread operates on a single array element (gene). It handles the entire array operations elementwise with a total of $popSize * length_of_individual$ operations running concurrently. Examples of this type include cost calculation, selection of parents, and crossover. The second type is a vector function where a single thread operates on the entire row of the array with a total of $popSize$ operations running concurrently. Although it operates in a parallel fashion on the entire array, it operates sequentially on each row. This type includes the kernels that require only one active thread in each warp (i.e., row) to access the entire chromosome such as: duplicate/missing nodes check, capacity constraints algorithm, and mutation.

It is worth mentioning that GPU functions support only numerical arrays with homogeneous widths. This property is not applicable in VRP problems because every single individual in the population array represents a possible solution for the problem and could have longer or shorter length than the others. To overcome this, all individuals are initialized with a fixed length of the longest possible solution. Additional genes inside shorter individuals are assigned to zero. Figure 3 illustrates the implementation of the proposed algorithm on the GPU, the outline of the algorithm described in Section 3.2 is detailed as follows:

Step 1: the program starts by reading the problem data file that includes a list of node IDs, their x and y coordinates, their demand values, and the vehicle capacity. Then, it saves them into a data array of size $n \times 4$ where n is the number of nodes. The pre-defined number of blocks and threads per block are used by the rest of the steps.

Step 2: the distance between every pair of nodes is represented by a hollow symmetric matrix (i.e., cost array) of size $n \times n$ is computed by a *cost* GPU kernel based on Equation 1. This array includes all the distances c_{ij} between each pair of nodes. The cost

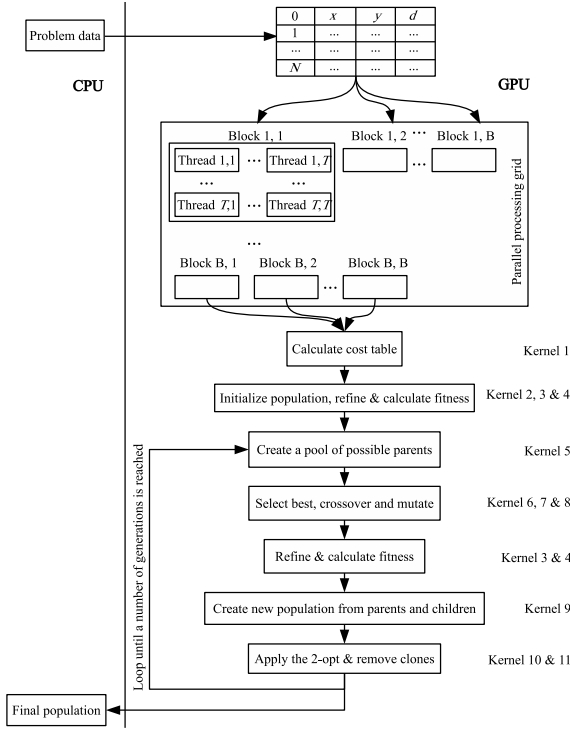


Figure 3: Algorithm implementation on GPU.

array is saved in the GPU global memory since it is too large to be copied to shared memories.

Step 3: the *initialize* kernel is called to randomly initialize a population array P_0 of size $popSize \times individual_length$ where $popSize$ is pre-selected and the individual length depends on the number of nodes. All random numbers are generated directly by the GPU using xoroshiro128+ pseudorandom number generator [7]. The reason of the random initialization of the population is to generate a pool of solutions that is spread across the entire search space [22]. Each row of P_0 represents a possible solution for the problem.

Step 4: *refining* kernels check for duplicate or missing nodes and insert depot nodes to count for capacity constraints if the cumulative customer demand values obtained by Equation 3 is larger than or equal to the vehicle capacity Q . Once the initial population is constructed, the evolution process begins.

Step 5: a *binary tournament* kernel is called to create a pool of four candidate arrays by randomly selecting individuals from P_0 . Each corresponding row in the four arrays represents a prospective parent in the following process. The best two individuals (the two with the lowest cost) from each corresponding row in the candidate arrays are selected to form two parent arrays.

Step 6: every corresponding row of the two parent arrays is used in a *crossover* kernel to form two child arrays. A standard k-point crossover is adopted where k is a static number for every single problem instance. This number is pre-selected from one to five according to the number of nodes in the problem instance.

Step 7: a random swap mutation process is applied to the child arrays by a *mutate* kernel. Two random genes in each individual are chosen to switch their locations with a pre-set occurrence (mutation) probability. Then, refining kernels are re-called to adjust the generated child arrays and relocate the depot nodes according to the capacity limits.

Step 8: *2-opt* local search kernel is applied to the child arrays to further improve the generated children. The cost of individuals is re-calculated to count for possible changes made through the entire evolution process, the 2-opt algorithm, and the refining process. Generated children enter the new population P_t only if they have a lower cost than their parents. This greedy replacement compares the children with their parents only, not with the entire population. If parents have very poor fitnesses and their children have better ones, the children will still be selected even if they have higher costs than some other parents. The adopted elitism concept compensates this issue to guarantee preserving high quality parents in the new population by carrying the top 5% individuals from the old population to the new one [15].

Step 9: to keep the diversity of the population, a *unique* kernel is used to remove duplicate individuals and replace them with others from parent and child arrays. The new population is then used for another evolution cycle until reaching a specific number of generations.

Finally, the best individual with the lowest cost in the final population is copied to the host (CPU) for program output. By utilizing these kernels, each thread on the GPU is assigned to work on one point in the chromosome which guarantees a parallel run of the proposed algorithm and hence an increase in the execution speed.

The pseudocodes shown in Algorithms 1 and 2 are example implementations of the cost calculating function on both CPU and GPU. Given the x, y coordinates of each node, the function calculates the distance c_{ij} between each pair and outputs an array of distances (costs).

Algorithm 1 CPU implementation of the cost calculating function

Require: $c_{ij} \forall i, j \in N$

```

1: function CALCOST_TBL(coord, cost_tbl)
2:   for index, node in enumerate(coord[:, 0]) do
3:     cost_tbl[index, index + 1] =
       np.round(np.hypot(np.subtract([coord[index, 2]]
         * len(coord[index + 1 : 2]), coord[index + 1 : 2]),
         np.subtract([coord[index, 3]] * len(coord[index + 1 :
           3]), coord[index + 1 : 3])))
4:   cost_tbl = np.add(cost_tbl, np.transpose(cost_tbl))
5:   end for
6:   return (cost_tbl)
7: end function

```

It is noticed from the GPU implementation how the input and output arrays (i.e., *coord* and *cost_tbl*) are distributed over the GPU grid. Line 2 gives a unique thread index within the entire grid. This thread will do the work on the array element which has the same index within the array. Line 3 is useful for arrays of size larger than the grid size. It enables the thread to have more

Algorithm 2 GPU implementation of the cost calculating function

Require: $c_{ij} \forall i, j \in N$

```

1: function CALCCOSTTBL(coord, cost_tbl)
2:   th_row, th_col = cuda.grid(2)
3:   stride_x, stride_y = cuda.gridsize(2)
4:   for row in range(th_row, coord.shape[0], stride_x) do
5:     for col in range(th_col, coord.shape[0], stride_y) do
6:       cost_tbl[row, col] =
         round(hypot(coord[row, 2] - coord[col, 2],
           coord[row, 3] - coord[col, 3]))
7:     end for
8:   end for
9: end function

```

than one unique index and stride over the grid using the *for* loops in Lines 4 and 5 to cover the entire array. This arrangement results in having each thread perform the same operation on parallel which, in turns, increases the execution speed of the algorithm. To compare between the CPU and the GPU implementations, both functions are executed for seven independent runs on a problem of 200 nodes. The results show that the average execution speed of the CPU is $8.81 \text{ ms} \pm 21.1 \mu\text{s}$ per loop whereas on the GPU it took $305 \mu\text{s} \pm 110 \mu\text{s}$ per loop of average speed. Despite the CPU version is also parallelized over the CPU cores, the GPU execution speed is 29 times faster. This is due to the special architecture that the GPU has which supports parallel intensive calculations as well as the limited number of cores the CPU has compared to the GPU. Interested users can refer to our Git repository for the source code: https://github.com/MarwanAbdelatti/GA_VRP_GPU.git

4 EXECUTION RESULTS

The proposed algorithm is coded using CUDA Python with Numba and CuPy library, and runs in CUDA 10.0 environment. The GPU used is an NVIDIA GeForce RTX 2080 that has a Turing architecture and 8 GB global memory, 2944 CUDA cores, and 1024 maximum number of threads per block. The number of blocks ($B \times B$) is 5×5 . In order to avoid 100% occupancy [42], each block is configured to have 400 (20×20) threads. The population size (*popSize*) is selected to be 100. These arrangements are suitable for problems with individuals of 100 points to be processed concurrently. For larger problems, a stride is used to project the device grid on the input arrays with a *stride_value* = $B * T = 100$. Theoretically, the algorithm should work on any GPU with Turing or Pascal architecture and 6 GB or more of shared memory. However, the only confirmed hardware we used is NVIDIA GeForce RTX 2080, GTX 1060 and GTX1070.

Table 1 summarizes the parameter settings. The number of generations and the number of crossover points depend on the problem size. Small problems require about 2000 generations to solve whereas large problems may require up to 10^6 generations. Similarly, the number of crossover points is selected by trying the algorithm with different numbers.

The proposed algorithm has been tested on 14 common benchmark CVRP problems selected from [2, 9, 10, 18, 25, 31]. For the sake of fairness and to come to accurate conclusions, these benchmark problems have been selected with different sizes ranging

Table 1: Parameter settings

Parameter	Value
<i>popSize</i>	100
number of generations	Range from 2000 to 10^6
Total number of blocks ($B \times B$)	5×5
Number of threads per block ($T \times T$)	20×20
Selection	Binary tournament
Crossover	k-point
Crossover probability	1
number of crossover points	Range from 1 to 5
mutation probability	1:15

from 16 to 76 nodes and different problem sets. However, work with larger problems is going on. For speed comparison purposes, an identical version of the proposed algorithm has been developed to entirely run on the CPU. Each problem set is run 10 times and the average is reported. A comparison between our results and the best-documented results is conducted. Moreover, we test the impact of the 2-opt local search on the proposed algorithm by running the algorithm for each problem set with and without the 2-opt. Two aspects have been considered to measure the performance of the proposed algorithm:

- (1) The execution speed which tracks the time elapsed by both algorithms to produce one population through the evolution processes.
- (2) The quality of solution where The %Gap [3] is used as a measure of quality for the obtained solution such that:

$$\%Gap = \frac{l_o - l^*}{l^*} \quad (4)$$

where l_o is the best solution obtained from our algorithm and l^* is the well-known published results from the literature. A good obtained solution is recognized by its low %Gap value. Table 2 shows the results of executing the algorithm (with the 2-opt included) on both CPU and GPU for the above-mentioned problems. The table shows the time elapsed to generate one population in seconds/loop, the best-known costs in the literature, and the best values obtained by the proposed algorithm (%Gap values).

It can be noticed from the table how the execution speed is significantly improved by using the GPU. While the elapsed execution time of the CPU is continuously increasing with the number of nodes, the elapsed time at the GPU is changing in a small range of values. The table shows speedups of the GPU version over the CPU by 52.7 times for the 21-node problem, 162.6 times for the 31-node problem, 368.6 times for the 40-node problem, 321.5 times for the 50-node problem, 339 times for the 60-node problem, and 454.4 times for the 76-node problem. It also shows how the speed gap between the CPU and the GPU implementations increases with the number of nodes.

The proposed algorithm has shown an adequate performance by reaching the best-known values of some problems typed in bold face and obtaining better values in others indicated by a boldface, negative %Gap value, and an asterisk. However, some high %Gap values are also obtained.

Table 2: Numerical results of testing CPU and GPU versions of the algorithm on benchmark problems

Problem ID	CPU time(sec/loop)	GPU time (sec/loop)	Best known	Obtained	Obtained %Gap (%)
P-n16-k8	0.79	0.0509	450	450	0
P-n20-k2	9.23	0.0850	216	216	0
P-n21-k2	5.37	0.1020	211	211	0
P-n22-k8	4.08	0.1060	603	590*	-2.16*
P-n23-k8	12.62	0.0821	529	529	0
B-n31-k5	20.86	0.1152	672	677	0.74
P-n40-k5	27.39	0.0743	458	478	4.37
B-n45-k5	31.14	0.0645	751	762	1.46
B-n50-k8	21.54	0.0670	1312	1327	1.14
P-n55-k15	36.00	0.0987	989	975*	-1.42*
P-n60-k15	34.51	0.1018	968	1061	9.61
P-n65-k10	25.34	0.1203	792	873	10.23
P-n70-k10	42.03	0.1312	827	892	7.86
E-n76-k14	48.80	0.1074	1021	1109	8.62

Figure 4 shows the algorithm execution time on both CPU and GPU. The logarithmic scale is meant to show the subtle differences in the results and to enable the plot to accept the large values resulted from the CPU. The figure shows that the execution time of the CPU keeps increasing with the problem size whereas the GPU execution time is oscillating around 0.1 sec/loop for all problem instances. This does not necessarily mean that the GPU execution time will be held around 0.1 sec/loop for any problem size.

We further investigate the impact of the 2-opt local search on the performance of the algorithm on the GPU. For each problem set, we exclude the 2-opt from the algorithm and examine the results. Table 3 shows the obtained value, the %Gaps, and the execution times of running the algorithm with and without the 2-opt. Reaching the best-known values is marked in bold face and obtaining better values is marked by a boldface, negative %Gap value, and an asterisk. It is obvious from the table the important role the 2-opt has in the algorithm. It is shown that taking out the 2-opt deteriorates the performance and negatively affects the solution quality. The %Gap obtained without the 2-opt is sometimes 5 times higher than that of having the 2-opt in the loop despite the reduced execution time. However, having longer execution time by involving the 2-opt is a feasible cost to pay for having a better solution quality in return, especially that increase is only 58.6 ms/loop at a maximum.

Figures 6 and 7 illustrate the results detailed in Table 3 and compare the performance of the algorithm with and without the 2-opt in terms of the solution quality and the execution time respectively.

5 CONCLUSIONS

This work proposes a new hybrid algorithm running on a GPU to take advantage of the special CUDA parallel processing. In this algorithm, a 2-opt local search is incorporated with a GA to solve capacitated VRP problems. The algorithm utilizes the GPU for handling the entire operations such as initializing population, reproduction, 2-opt local search, and the refining processes. A clone-restricting process is also adopted to maintain the diversity of the population.

In order to conduct a precise experiment, two versions of the algorithm have been applied to the same problem instances: one

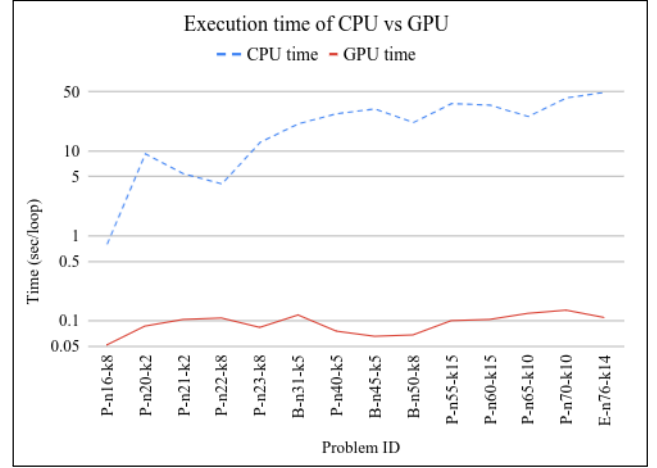
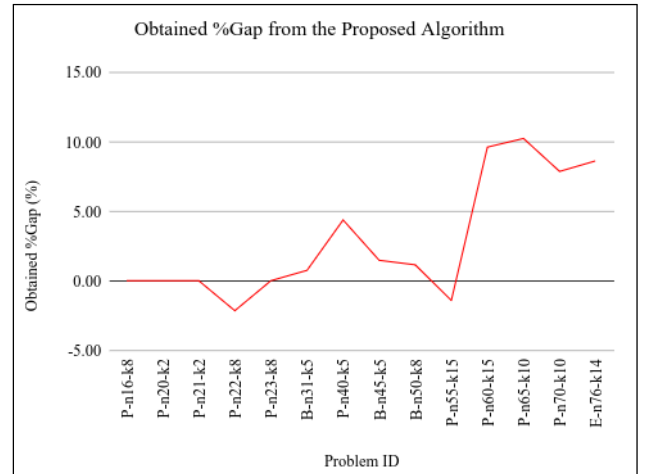
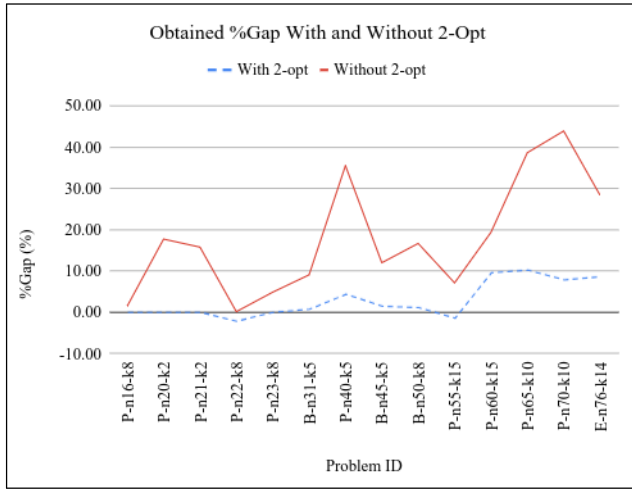
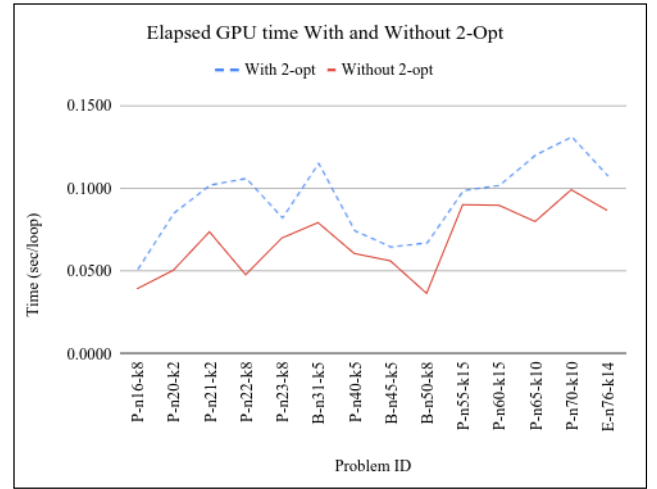
**Figure 4: Execution time of CPU vs GPU.****Figure 5: Solution quality of the proposed algorithm**

Table 3: Numerical results of running the GPU algorithm with and without the 2-opt

Problem ID	Best known	With 2-opt			Without 2-opt		
		Obtained value	Obtained %Gap (%)	Time (sec/loop)	Obtained value	Obtained %Gap (%)	Time (sec/loop)
P-n16-k8	450	450	0.00	0.0509	456	1.33	0.0389
P-n20-k2	216	216	0.00	0.0850	254	17.59	0.0501
P-n21-k2	211	211	0.00	0.1020	244	15.64	0.0734
P-n22-k8	603	590*	-2.16	0.1060	603	0.00	0.0474
P-n23-k8	529	529	0.00	0.0821	554	4.73	0.0696
B-n31-k5	672	677	0.74	0.1152	732	8.93	0.0790
P-n40-k5	458	478	4.37	0.0743	620	35.37	0.0603
B-n45-k5	751	762	1.46	0.0645	840	11.85	0.0558
B-n50-k8	1312	1327	1.14	0.0670	1529	16.54	0.0361
P-n55-k15	989	975*	-1.42	0.0987	1058	6.98	0.0898
P-n60-k15	968	1061	9.61	0.1018	1154	19.21	0.0895
P-n65-k10	792	873	10.23	0.1203	1097	38.51	0.0796
P-n70-k10	827	892	7.86	0.1312	1189	43.77	0.0989
E-n76-k14	1021	1109	8.62	0.1074	1309	28.21	0.0862

**Figure 6: Performance With and Without the 2-opt****Figure 7: Execution time with and without the 2-opt**

runs on the CPU and the other on the GPU. Two performance measures have been considered; (1) the execution time in seconds per loop; and (2) the %Gap value between the obtained costs and the best-known values. Corresponding to the first measure, a significant improvement in the execution speed has been observed. Unlike the exponential increase in CPU execution time, the GPU execution time is very low and almost constant. This is helpful when dealing with larger problem instances and when using a large number of generations for reaching higher-quality solutions in a feasible time. Considering the second measure, the proposed algorithm reached the best-known values of some problems and improved some others. High values are obtained for some problems. To study the 2-opt effect, the algorithm is run with and without it. The results show a significant impact of the 2-opt despite the increase in the execution time which is acceptable compared to the obtained improvements.

Our contributions in this work can be summarized as: (1) we address the execution speed problem in VRPs by developing an algorithm that is entirely executed on a GPU; (2) we investigate the utilization of a local search algorithm (i.e., the 2-opt) with the GA; (3) we improve some best-documented values of VRP benchmark problems; and (4) provide a detailed numerical analysis and comparison plots for the algorithm performance. Testing the proposed algorithm with larger problems is ongoing on. Further investigations will be conducted to improve the performance of the proposed algorithm to obtain higher-quality solutions. This will include different crossover operators like Best Cost-Best Route and Cyclical crossover, different mutation operators, and using modified GA algorithms like Cellular GA (cGAs) and Island model GAs for instance.

REFERENCES

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*. 265–283.
- [2] Ph Augerat, Jose Manuel Belenguer, Enrique Benavent, A Corberán, D Naddef, and G Rinaldi. 1995. *Computational results with a branch and cut code for the capacitated vehicle routing problem*. Vol. 34. IMAG.
- [3] Barrie M Baker and MA Ayeche. 2003. A genetic algorithm for the vehicle routing problem. *Computers & Operations Research* 30, 5 (2003), 787–800.
- [4] Roberto Baldacci, Nicos Christofides, and Aristide Mingozzi. 2008. An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Mathematical Programming* 115, 2 (2008), 351–385.
- [5] Richard J Bauer Jr, Richard J Bauer, et al. 1994. *Genetic algorithms and investment strategies*. Vol. 19. John Wiley & Sons.
- [6] A Benaini and A Berrajaa. 2018. Genetic algorithm for large dynamic vehicle routing problem on GPU. In *2018 4th International Conference on Logistics Operations Management (GOL)*. IEEE, 1–9.
- [7] David Blackman and Sebastiano Vigna. 2018. Scrambled linear pseudorandom number generators. *arXiv preprint arXiv:1805.01407* (2018).
- [8] John Cheng, Max Grossman, and Ty McKercher. 2014. *Professional CUDA c programming*. John Wiley & Sons.
- [9] Nicos Christofides and Samuel Eilon. 1969. An algorithm for the vehicle-dispatching problem. *Journal of the Operational Research Society* 20, 3 (1969), 309–318.
- [10] Nicos Christofides, Aristide Mingozzi, and Paolo Toth. 1981. Exact algorithms for the vehicle routing problem, based on spanning tree and shortest path relaxations. *Mathematical programming* 20, 1 (1981), 255–282.
- [11] Paul C Chu and John E Beasley. 1997. A genetic algorithm for the generalised assignment problem. *Computers & Operations Research* 24, 1 (1997), 17–23.
- [12] Geoff Clarke and John W Wright. 1964. Scheduling of vehicles from a central depot to a number of delivery points. *Operations research* 12, 4 (1964), 568–581.
- [13] IM Coelho, PLA Munhoz, LS Ochi, MJF Souza, C Bentes, and R Farias. 2016. An integrated CPU–GPU heuristic inspired on variable neighbourhood search for the single vehicle routing problem with deliveries and selective pickups. *International Journal of Production Research* 54, 4 (2016), 945–962.
- [14] George B Dantzig and John H Ramser. 1959. The truck dispatching problem. *Management science* 6, 1 (1959), 80–91.
- [15] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [16] Samuel Eilon, Carl Donald Tyndale Watson-Gandy, Nicos Christofides, and Richard de Neufville. 1974. Distribution management-mathematical modelling and practical analysis. *IEEE Transactions on Systems, Man, and Cybernetics* 6 (1974), 589–589.
- [17] Nasser A El-Sherbeny. 2010. Vehicle routing with time windows: An overview of exact, heuristic and metaheuristic methods. *Journal of King Saud University-Science* 22, 3 (2010), 123–131.
- [18] Marshall L Fisher. 1994. Optimal solution of vehicle routing problems using minimum k-trees. *Operations research* 42, 4 (1994), 626–642.
- [19] Marshall L Fisher and Ramchandran Jaikumar. 1981. A generalized assignment heuristic for vehicle routing. *Networks* 11, 2 (1981), 109–124.
- [20] Abel Garcia-Najera and John A Bullinaria. 2008. A multi-objective density restricted genetic algorithm for the vehicle routing problem with time windows. In *2008 UK Workshop on Computational Intelligence*. 47–52.
- [21] Abel Garcia-Najera and John A Bullinaria. 2009. Comparison of similarity measures for the multi-objective vehicle routing problem with time windows. In *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*. 579–586.
- [22] Abel Garcia-Najera and John A Bullinaria. 2011. An improved multi-objective evolutionary algorithm for the vehicle routing problem with time windows. *Computers & Operations Research* 38, 1 (2011), 287–300.
- [23] Michel Gendreau, Alain Hertz, and Gilbert Laporte. 1994. A tabu search heuristic for the vehicle routing problem. *Management science* 40, 10 (1994), 1276–1290.
- [24] Aurélien Géron. 2017. *Hands-on machine learning with Scikit-Learn and TensorFlow: concepts, tools, and techniques to build intelligent systems*. " O'Reilly Media, Inc".
- [25] Billy E Gillett and Jerry G Johnson. 1976. Multi-terminal vehicle-dispatch algorithm. *Omega* 4, 6 (1976), 711–718.
- [26] Billy E Gillett and Leland R Miller. 1974. A heuristic algorithm for the vehicle-dispatch problem. *Operations research* 22, 2 (1974), 340–349.
- [27] Nikhil Ketkar. 2017. Introduction to pytorch. In *Deep learning with python*. Springer, 195–208.
- [28] Nagesh B Lakshminarayana and Hyesoon Kim. 2010. Effect of instruction fetch and memory scheduling on gpu performance. In *Workshop on Language, Compiler, and Architecture Support for GPGPU*, Vol. 88. Citeseer.
- [29] Gilbert Laporte. 1992. The vehicle routing problem: An overview of exact and approximate algorithms. *European journal of operational research* 59, 3 (1992), 345–358.
- [30] Gilbert Laporte and Yves Nobert. 1987. Exact algorithms for the vehicle routing problem. In *North-Holland Mathematics Studies*. Vol. 132. Elsevier, 147–184.
- [31] Feiyue Li, Bruce Golden, and Edward Wasil. 2005. Very large-scale vehicle routing: new test problems, algorithms, and results. *Computers & Operations Research* 32, 5 (2005), 1165–1179.
- [32] Jianming Li, Ximeng Lv, and Linlin Liu. 2011. A parallel genetic algorithm with GPU accelerated for large-scale MDVRP in emergency logistics. In *2011 14th IEEE International Conference on Computational Science and Engineering*. IEEE, 602–605.
- [33] Zhichao Lu, Ian Whalen, Vishnu Boddeti, Yashesh Dhebar, Kalyanmoy Deb, Erik Goodman, and Wolfgang Banzhaf. 2019. NSGA-Net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 419–427.
- [34] Paulo Mendes. 2011. *Demand driven supply chain: A structured and practical roadmap to increase profitability*. Springer Science & Business Media.
- [35] Jean-Yves Potvin and Samy Bengio. 1996. The vehicle routing problem with time windows part II: genetic search. *INFORMS journal on Computing* 8, 2 (1996), 165–172.
- [36] Jean-Yves Potvin and Jean-Marc Rousseau. 1995. An exchange heuristic for routing problems with time windows. *Journal of the Operational Research Society* 46, 12 (1995), 1433–1446.
- [37] Christian Prins. 2004. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research* 31, 12 (2004), 1985–2002.
- [38] Jason Sanders and Edward Kandrot. 2010. *CUDA by example: an introduction to general-purpose GPU programming*. Addison-Wesley Professional.
- [39] Il Schmitt and Lawrence John. 1994. An empirical computational study of genetic algorithms to solve order based problems: an emphasis on TSP and VRPTC. (1994).
- [40] LJ Schmitt. 1995. *An evaluation of a genetic algorithmic approach to the VRP*. Technical Report. Working paper.
- [41] Kenneth Sörensen. 2007. Distance measures based on the edit distance for permutation-type representations. *Journal of Heuristics* 13, 1 (2007), 35–47.
- [42] Duane Storti and Mete Yurtoglu. 2015. *CUDA for engineers: an introduction to high-performance parallel computing*. Addison-Wesley Professional.
- [43] Jagielło Szymon and Żelazny Dominik. 2013. Solving multi-criteria vehicle routing problem by parallel tabu search on GPU. *Procedia Computer Science* 18 (2013), 2529–2532.
- [44] Paolo Toth and Daniele Vigo. 2002. *The vehicle routing problem*. SIAM.
- [45] Min Wen, Jens Clausen, and Jesper Larsen. 2010. Rich vehicle routing problems and applications. (2010).
- [46] Mieczysław Wodecki, Wojciech Bożejko, Szymon Jagielło, and Jarosław Pempera. 2015. Parallel Cost Function Determination on GPU for the Vehicle Routing Problem. In *International Conference on Artificial Intelligence and Soft Computing*. Springer, 778–788.